

Událostmi řízené programování

Co je událostmi řízené programování

- **událostmi řízené programování** = programovací přístup, kde běh programu řídí události
- program neprovádí jen pevně daný postup od začátku do konce
- program čeká, až nastane nějaká událost
- po vzniku události se spustí příslušná reakce

Událost může být například:

- kliknutí na tlačítko
- stisk klávesy
- pohyb myši
- změna hodnoty ve formuláři
- příchozí zpráva
- dokončení načítání souboru
- vypršení časovače

Typické použití

- grafická uživatelská rozhraní
- webové aplikace
- formuláře
- hry
- mobilní aplikace
- aplikace pracující se vstupy uživatele
- asynchronní zpracování požadavků

Příklad:

- uživatel klikne na tlačítko
- vznikne událost `Click`
- program spustí metodu, která na kliknutí reaguje

Základní pojmy

Event

- **event** = událost
- signál, že se něco stalo
- na základě události se může spustit určitý kód

- v C# se používá klíčové slovo `event`

Příklady eventů:

- `Click`
- `TextChanged`
- `KeyDown`
- `MouseMove`
- `Loaded`

Publisher

- **publisher** = zdroj události
- objekt, ve kterém událost vzniká
- někdy se mu říká také **vydavatel**
- oznamuje ostatním objektům, že se něco stalo

Příklad:

- tlačítko je publisher události `Click`
- časovač je publisher události `Tick`
- formulář může být publisher události `Loaded`

Subscriber

- **subscriber** = odběratel události
- objekt nebo metoda, která na událost reaguje
- přihlásí se k odběru události
- po vyvolání události se spustí jeho obslužná metoda

Příklad:

- metoda `Button_Click` je subscriber události `Click`
- formulář může odebírat událost tlačítka

Handler

- **event handler** = obslužná metoda události
- metoda, která se spustí, když událost nastane
- obsahuje reakci na událost

Příklad:

```
void Button_Click(object sender, EventArgs e)
{
```

```
Console.WriteLine("Kliknuto na tlačítko.");  
}
```

Návrhový vzor Observer

- událostmi řízené programování často vychází ze vzoru **Observer**
- česky se mu říká **pozorovatel**
- jeden objekt sleduje změny jiného objektu
- když se stav objektu změní, ostatní objekty jsou upozorněny

Princip Observeru

- existuje objekt, který událost vyvolává
- tento objekt se nazývá **publisher**
- existují objekty, které chtějí být upozorněny
- tyto objekty se nazývají **subscribers**
- subscriber se přihlásí k odběru události
- publisher při vzniku události upozorní všechny přihlášené subscribery

Příklad:

```
Publisher → Event → Subscribers  
Tlačítko → Click → Button_Click(), LogClick(), UpdateUI()
```

Publisher-Subscriber princip

Publisher

- událost vytváří
- udržuje seznam přihlášených handlerů
- při vzniku události zavolá přihlášené handlersy
- neví přesně, co jednotliví odběratelé udělají
- pouze oznámí, že událost nastala

Subscriber

- přihlašuje se k události
- poskytuje metodu, která se má spustit
- reaguje na událost
- může se z odběru odhlásit

Výhoda:

- publisher a subscriber jsou volněji propojení

- publisher nemusí znát konkrétní logiku subscriberu
- lze snadno přidat více reakcí na jednu událost

Delegát v Csharp (C#)

- **delegát** = typ, který určuje tvar metody
- říká:
 - jaké parametry má metoda mít
 - jaký návratový typ má mít
- delegát může držet odkaz na jednu nebo více metod
- používá se jako základ pro události

Jednoduše:

- delegát je „proměnná na metodu“
- event potom používá delegát pro určení, jaké metody se mohou přihlásit

Příklad delegátu

```
delegate void MujHandler(string zprava);
```

- metoda odpovídající tomuto delegátu musí:
 - nic nevracet
 - přijímat jeden parametr typu `string`

Příklad metody:

```
void Vypis(string text)
{
    Console.WriteLine(text);
}
```

Event v Csharp (C#)

- **event** = obal nad delegátem
- umožňuje bezpečné přihlašování a odhlašování handlerů
- mimo třídu lze obvykle použít pouze:
 - `+=` pro přihlášení
 - `-=` pro odhlášení
- event může vyvolat pouze třída, která ho definuje

Výhoda:

- chrání událost před nechtěným přepsáním zvenku

- chrání zapouzdření
- subscriber nemůže zvenku událost libovolně spustit

Typický tvar handleru v Csharp (C#)

- V C# se často používá tvar:

```
void Metoda(object sender, EventArgs e)
{
    // reakce na událost
}
```

object sender

- objekt, který událost vyvolal
- tedy publisher
- například tlačítko, formulář nebo časovač

EventArgs e

- data události
- může obsahovat další informace
- například:
 - souřadnice myši
 - stisknutou klávesu
 - novou hodnotu
- pokud událost žádná data neposílá, používá se `EventArgs.Empty`

Přihlášení a odhlášení z události

Přihlášení

- subscriber se přihlásí pomocí operátoru `+=`
- tím se handler přidá do seznamu metod, které se mají zavolat

Příklad:

```
button.Click += Button_Click;
```

Odhlášení

- subscriber se odhlásí pomocí operátoru `-=`
- tím se handler odebere ze seznamu

Příklad:

```
button.Click -= Button_Click;
```

Vlastní event v Csharp (C#)

1. Vytvoření publisheru

```
class Tlacitko
{
    public event EventHandler Kliknuto;

    public void Klik()
    {
        Console.WriteLine("Tlačítko bylo stisknuto.");

        if (Kliknuto != null)
        {
            Kliknuto(this, EventArgs.Empty);
        }
    }
}
```

Vysvětlení:

- `Tlacitko` je publisher
- `Kliknuto` je event
- metoda `Klik()` událost vyvolá
- `this` označuje objekt, který událost vyvolal
- `EventArgs.Empty` znamená, že neposíláme žádná další data

2. Vytvoření subscriberu

```
class Program
{
    static void Main()
    {
        Tlacitko tlacitko = new Tlacitko();

        tlacitko.Kliknuto += ReakceNaKlik;

        tlacitko.Klik();
    }

    static void ReakceNaKlik(object sender, EventArgs e)
    {
        Console.WriteLine("Subscriber reaguje na kliknutí.");
    }
}
```

```
}  
}
```

Vysvětlení:

- Program se přihlásí k odběru události
- `ReakceNaKlik` je handler
- po zavolání `tlacitko.Klik()` se handler spustí

3. Odhlášení z eventů

```
tlacitko.Kliknuto -= ReakceNaKlik;
```

- po odhlášení se metoda `ReakceNaKlik` už nespustí

Bezpečné vyvolání eventů

- Místo kontroly `if (Kliknuto != null)` se často používá kratší zápis:

```
Kliknuto?.Invoke(this, EventArgs.Empty);
```

- `?.` znamená, že se event vyvolá jen tehdy, pokud není `null`
- `Invoke` spustí všechny přihlášené handlers

Upravený příklad:

```
class Tlacitko  
{  
    public event EventHandler Kliknuto;  
  
    public void Klik()  
    {  
        Console.WriteLine("Tlačítko bylo stisknuto.");  
        Kliknuto?.Invoke(this, EventArgs.Empty);  
    }  
}
```

Event s vlastními daty

- někdy potřebujeme s událostí poslat i vlastní data
- vytvoříme vlastní třídu dědící z `EventArgs`

Příklad:

```

class TeplotaEventArgs : EventArgs
{
    public int Teplota { get; }

    public TeplotaEventArgs(int teplota)
    {
        Teplota = teplota;
    }
}

```

Publisher:

```

class Senzor
{
    public event EventHandler<TeplotaEventArgs> TeplotaZmenena;

    public void NastavTeplotu(int teplota)
    {
        TeplotaZmenena?.Invoke(this, new TeplotaEventArgs(teplota));
    }
}

```

Subscriber:

```

class Program
{
    static void Main()
    {
        Senzor senzor = new Senzor();

        senzor.TeplotaZmenena += VypisTeplotu;

        senzor.NastavTeplotu(25);
    }

    static void VypisTeplotu(object sender, TeplotaEventArgs e)
    {
        Console.WriteLine("Nová teplota: " + e.Teplota);
    }
}

```

Eventy v JavaScriptu

- v JavaScriptu se s eventy pracuje hlavně ve webových aplikacích
- typický příklad je `addEventListener`

Příklad:

```
button.addEventListener("click", function (e) {  
    console.log("Kliknuto!");  
});
```

Nebo pomocí lambda / arrow funkce:

```
button.addEventListener("click", e => {  
    console.log("Kliknuto!");  
});
```

Vysvětlení:

- `button` je publisher
- `"click"` je event
- funkce uvnitř je handler / subscriber
- parametr `e` obsahuje data události

Příklady běžných eventů

Desktopové aplikace

- `Click`
- `MouseMove`
- `KeyDown`
- `TextChanged`
- `Loaded`
- `Closing`

Webové aplikace

- `click`
- `submit`
- `change`
- `input`
- `keydown`
- `load`

Výhody událostmi řízeného programování

- vhodné pro aplikace reagující na uživatele
- dobré oddělení zdroje události a reakce
- jedna událost může mít více subscriberů
- subscriber se může přihlásit i odhlásit

- program nemusí neustále aktivně kontrolovat, jestli se něco stalo
- dobře se používá v GUI a webových aplikacích

Nevýhody / rizika

- tok programu může být méně přehledný
- složitější ladění
- může být těžší sledovat, co se kdy spustí
- zapomenuté odhlášení může někdy způsobit problémy s pamětí
- mnoho eventů může vést k nepřehlednému kódu